```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report, confusion_ma
```

```
from datasets import load_dataset

dataset = load_dataset("imdb")
df = dataset['train'].to_pandas()

df = df[['text', 'label']]
df.head()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserW
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
```

README.md: 7.81k/? [00:00<00:00, 555kB/s]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
```

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:01<00:00, 105MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:01<00:00, 68.5MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 66087.08 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 72908.54 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 86661.76 examples/s]

	text	label
0	I rented I AM CURIOUS-YELLOW from my video sto...	0
1	"I Am Curious: Yellow" is a risible and preten...	0
2	If only to avoid making this type of film in t...	0
3	This film was probably inspired by Godard's Ma...	0
4	Oh, brother...after hearing about this ridicul...	0

Next steps: Generate code with df New interactive sheet

```
df = df.dropna()

df['label'] = df['label'].astype(int)
```

```
X = df['text']
y = df['label']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape)
```

```
(20000,) (5000,)
```

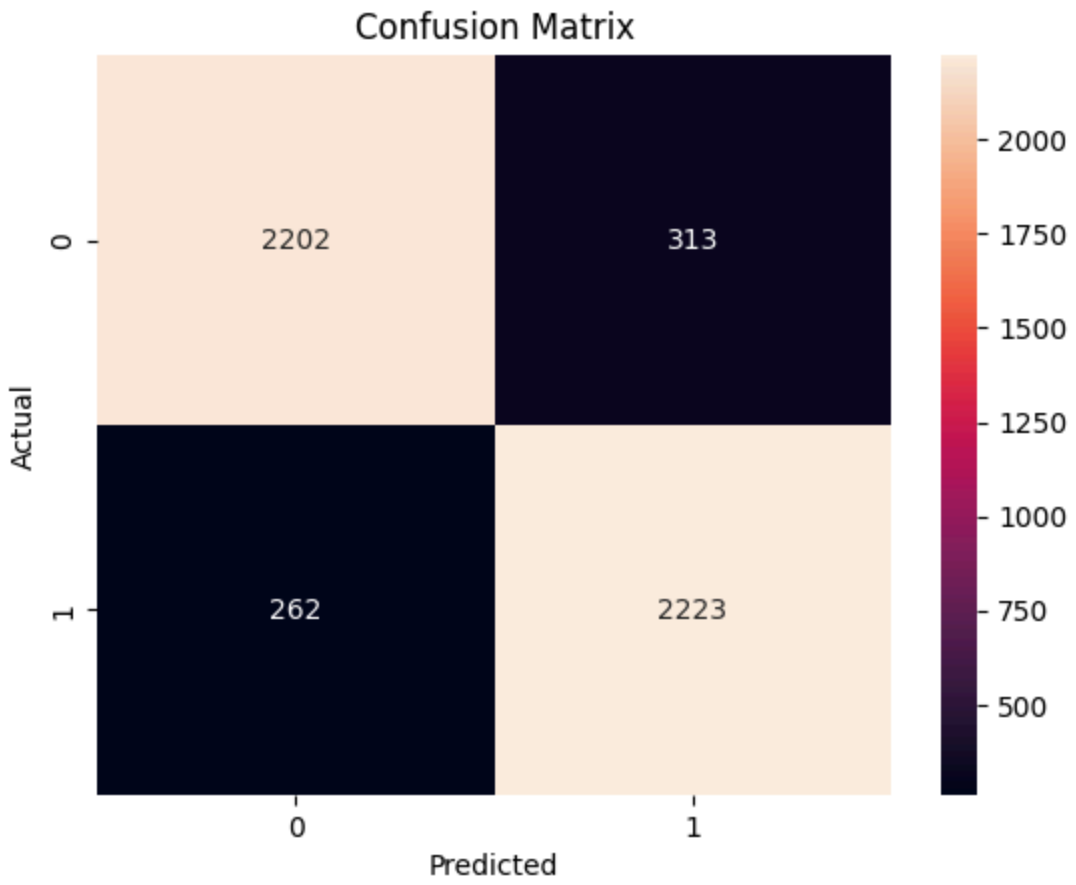
```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_tfidf, y_train)

y_pred = lr.predict(X_test_tfidf)
```

```
from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.885
              precision    recall  f1-score   support

           0       0.89      0.88      0.88      2515
           1       0.88      0.89      0.89      2485

    accuracy                           0.89      5000
   macro avg       0.89      0.89      0.88      5000
weighted avg       0.89      0.89      0.88      5000
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



The model performs well with a high number of correct classifications for both positive and negative sentiments. The confusion matrix shows balanced performance with relatively low false positives and false negatives, indicating the model generalizes well.

```
from sklearn.svm import LinearSVC

svm = LinearSVC()
svm.fit(X_train_tfidf, y_train)

y_pred_svm = svm.predict(X_test_tfidf)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM Accuracy: 0.8802
              precision    recall  f1-score   support

           0       0.89      0.87      0.88      2515
           1       0.87      0.89      0.88      2485

    accuracy                           0.88      5000
   macro avg       0.88      0.88      0.88      5000
weighted avg       0.88      0.88      0.88      5000
```

```
from sklearn.svm import LinearSVC

svm = LinearSVC()
svm.fit(X_train_tfidf, y_train)

y_pred_svm = svm.predict(X_test_tfidf)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM Accuracy: 0.8802
              precision    recall  f1-score   support

           0       0.89      0.87      0.88      2515
           1       0.87      0.89      0.88      2485

    accuracy                           0.88      5000
   macro avg       0.88      0.88      0.88      5000
weighted avg       0.88      0.88      0.88      5000
```

The Support Vector Machine (SVM) model achieved an accuracy of 88%, demonstrating strong performance in classifying sentiment. The precision and recall scores are well balanced across both classes, indicating that the model performs consistently for both positive and negative reviews

The SVM model slightly outperformed Logistic Regression, likely due to its ability to handle high-dimensional text data more effectively.

```
!pip install transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/di
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/pyt
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (
```

```
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pack
```

```
from transformers import pipeline

bert = pipeline("sentiment-analysis")

bert("This movie was amazing!")
bert("This movie was terrible.")
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned
Using a pipeline without specifying a model name and revision in production is n
```

config.json: 100% 629/629 [00:00<00:00, 23.3kB/s]

model.safetensors: 100% 268M/268M [00:01<00:00, 226MB/s]

Loading weights: 100% 104/104 [00:00<00:00, 135.48it/s, Materializing param=pre_classifier.weight]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 812B/s]